PsiTwin Inventory

Guía funcional del backend para desarrollo del frontend

Este documento explica, en lenguaje práctico, qué representa cada entidad del sistema, cómo se relacionan entre sí y qué endpoint debe consumir el frontend en cada acción. La idea es que una persona que no haya participado en el desarrollo del backend pueda entender el flujo general y comenzar a construir las pantallas y formularios.

Base local utilizada en los ejemplos

http://127.0.0.1:8000/api/v1

Documento de integración funcional

1. Cómo entender el sistema

PsiTwin Inventory organiza la información de un almacén desde lo más general hasta lo más específico. Una empresa puede tener uno o varios almacenes; cada almacén se divide en zonas y cada zona contiene racks. Los productos se ubican en esos racks y el inventario indica cuánto stock real existe en cada ubicación. A partir de allí, los movimientos registran cada entrada, salida o ajuste para conservar el historial de cambios.

Además del control diario de stock, el sistema incluye un flujo de reposición. Cuando se necesita abastecer productos, se trabaja con proveedores y órdenes de reposición. La orden contiene uno o varios productos solicitados. Cuando el proveedor entrega la mercadería, se registra una recepción y se indica cuánto llegó correctamente y cuánto fue rechazado. El backend genera automáticamente el movimiento de entrada correspondiente y actualiza el inventario.

Para el frontend, esto significa que no todas las pantallas trabajan de forma aislada: muchas dependen de datos creados en pasos anteriores. Por ejemplo, para crear un inventario ya debe existir el producto y el rack; para registrar una recepción ya debe existir una orden enviada; y para registrar el detalle de una recepción ya debe existir un inventario donde ingresar las unidades aceptadas.

Autenticación y uso general

La empresa se registra e inicia sesión mediante los endpoints de autenticación. El login devuelve un token que el frontend debe conservar y enviar en las peticiones protegidas. En la práctica, el frontend suele guardar el token mientras la sesión esté activa y agregarlo en el encabezado Authorization como Bearer token.

```
Authorization: Bearer <TOKEN>
```

La documentación interactiva del backend puede consultarse en **http://127.0.0.1:8000/docs**. También existe un endpoint de salud en **http://127.0.0.1:8000/health**.

2. Flujo general del sistema

El flujo empieza con la empresa. La empresa crea sus almacenes; cada almacén se divide en zonas y dentro de las zonas se crean los racks. Después se registran los productos y se crea el inventario que vincula un producto con un rack y define su stock actual, mínimo y máximo. A partir de ese momento, cualquier cambio real de existencias se registra como un movimiento. Si se trata de una entrada, el stock aumenta; si se trata de una salida, disminuye; y si se trata de un ajuste, la cantidad enviada representa el nuevo stock físico contado.

El operario es la persona que participa en las operaciones del almacén. Está vinculado a una empresa y a un almacén. Cuando se registra un movimiento manual, se indica qué inventario fue afectado y qué operario realizó la acción. El movimiento conserva el stock anterior y el stock nuevo, por lo que funciona como historial y no debe editarse ni eliminarse desde el frontend.

El segundo flujo importante es la reposición. Primero se registra un proveedor. Luego se crea una orden de reposición indicando la empresa, el almacén y el proveedor. Mientras la orden está PENDIENTE, se agregan los productos y cantidades que se desean solicitar. Cuando la orden está lista, se aprueba y después se marca como enviada. Recién entonces se puede registrar la recepción de mercadería.

Al recibir el pedido, el frontend crea la recepción indicando la orden y el operario que recibe. Después registra uno o más detalles de recepción. En cada detalle se indica qué línea de la orden se está recibiendo, a qué inventario ingresará el producto, cuántas unidades llegaron y cuántas fueron rechazadas. El backend calcula automáticamente la cantidad aceptada. Si hay unidades aceptadas, crea un movimiento de tipo ENTRADA con motivo RECEPCION y aumenta el stock del inventario. Por eso el frontend no debe crear un segundo movimiento manual para la misma recepción.

La orden se actualiza según lo recibido. Si solo se recibió una parte de lo solicitado, queda en estado PARCIAL. Cuando la suma de las unidades aceptadas alcanza todo lo solicitado, pasa a RECIBIDA. Así, el estado de la orden refleja el avance real de la recepción y no depende de un cambio manual del usuario.

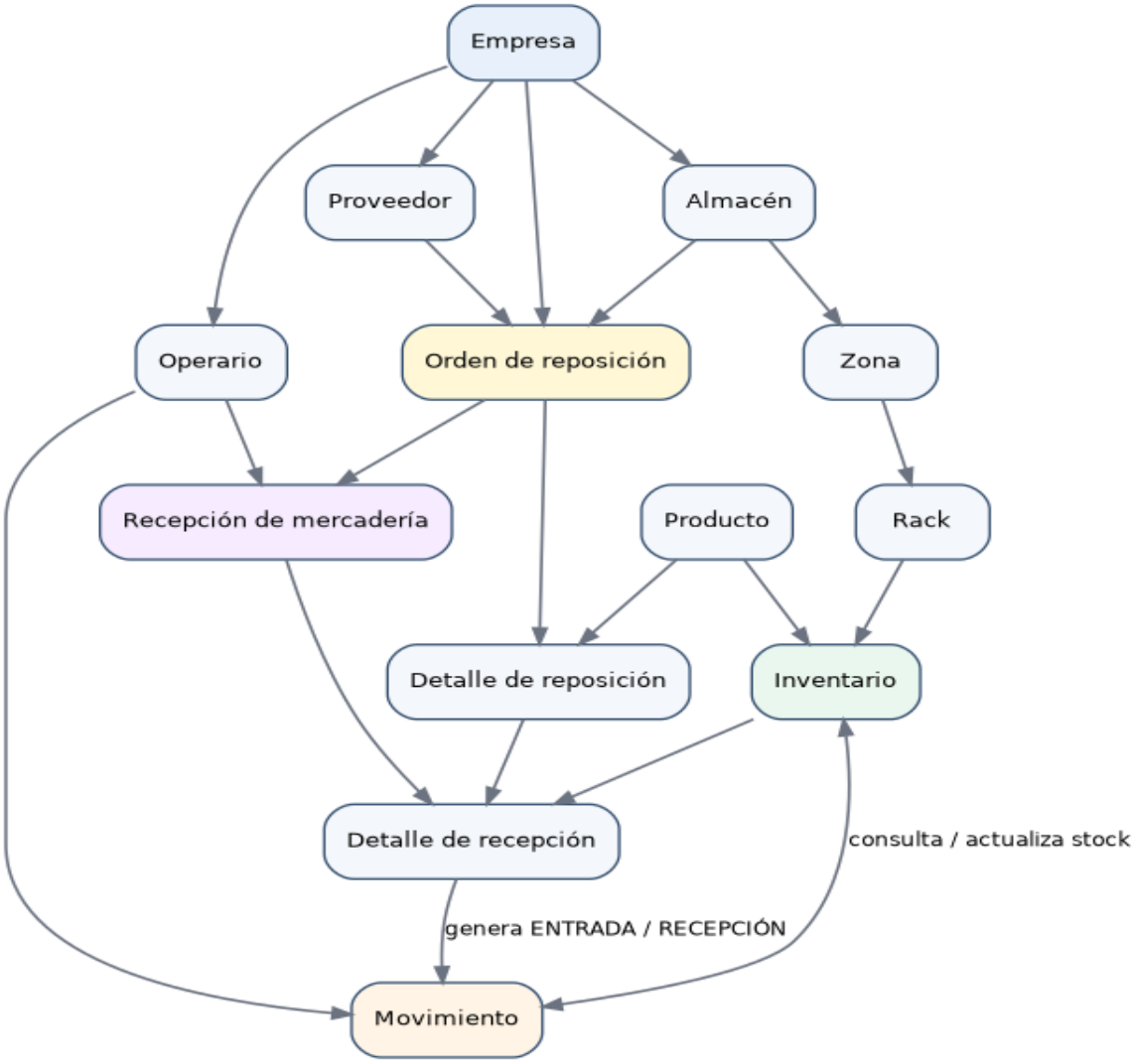


Figura: relación funcional entre las entidades principales y el flujo de reposición/recepción.

3. Definición de las entidades

Empresa

Representa a la organización que utiliza PsiTwin. Es el punto de partida del sistema porque los almacenes, productos, operarios y proveedores pertenecen a una empresa. También contiene los datos de cuenta necesarios para registrar, iniciar sesión y administrar el perfil de la organización.

Almacén

Representa una ubicación física de almacenamiento perteneciente a una empresa. Guarda información como nombre, ubicación, área, aforo, tipo de almacén y coordenadas. Sirve como nivel principal para organizar zonas, operarios y órdenes de reposición.

Zona

Representa una división interna de un almacén. Puede utilizarse para separar áreas por función, capacidad o condiciones de almacenamiento. Una zona pertenece a un único almacén y contiene uno o varios racks.

Rack

Representa una estructura o ubicación física dentro de una zona. Es el nivel en el que finalmente se ubican los productos mediante el inventario. También puede guardar información de nivel, capacidad y estado, lo que permite al frontend mostrar si una posición está disponible, llena o en mantenimiento.

Producto

Representa el catálogo de artículos administrados por la empresa. Contiene datos como nombre, categoría, familia, marca, peso, volumen, condiciones de refrigeración, temperatura recomendada, tiempo de reposición y vida útil. Aunque existe un campo stock por compatibilidad, el stock operativo debe leerse principalmente desde Inventario.

Inventario

Representa el stock actual de un producto en un rack específico. Contiene el stock actual, mínimo y máximo. Es la referencia que debe utilizar el frontend para mostrar existencias reales y alertas como stock bajo, sin stock o sobre stock. Cada movimiento modifica este registro.

Operario

Representa a una persona que trabaja en un almacén. Está asociado a una empresa y a un almacén, y contiene datos como nombre, apellido, cargo, turno y estado. Para registrar recepciones o movimientos se utiliza el operario responsable de la operación.

Movimiento

Representa el historial de cambios de stock. Puede ser ENTRADA, SALIDA o AJUSTE y además registra el motivo, por ejemplo RECEPCION, VENTA, MERMA o CONTEO_FISICO. Guarda el stock anterior y el nuevo para que pueda auditarse qué ocurrió. Debe tratarse como un historial inmutable: el frontend lo consulta y crea cuando corresponde, pero no lo edita ni elimina.

Proveedor

Representa a la empresa o persona que abastece productos. Se relaciona con la empresa propietaria del sistema y almacena nombre, RUC, correo, teléfono, dirección, tiempo estimado de entrega y estado. Es necesario tener un proveedor activo antes de crear una orden de reposición.

Orden de reposición

Representa una solicitud de abastecimiento enviada a un proveedor para un almacén. Empieza como PENDIENTE, puede aprobarse y enviarse, y luego pasa a PARCIAL o RECIBIDA según las recepciones registradas. También puede cancelarse antes del envío. La orden funciona como cabecera y sus productos se almacenan en DetalleReposicion.

Detalle de reposición

Representa una línea dentro de una orden de reposición. Indica qué producto se está solicitando, cuántas unidades se necesitan y, si corresponde, el costo unitario. Solo debe modificarse mientras la orden permanezca PENDIENTE.

Recepción de mercadería

Representa el momento en que llega una orden al almacén. Está vinculada a la orden y al operario que recibe. Puede existir más de una recepción para una misma orden cuando el proveedor entrega en partes. Solo se registra cuando la orden está ENVIADA o PARCIAL.

Detalle de recepción

Representa lo que efectivamente llegó de una línea de la orden. Registra cantidad recibida, cantidad rechazada y cantidad aceptada, además del inventario donde entrarán las unidades. Si existe cantidad aceptada, el backend genera automáticamente el movimiento de entrada y actualiza el inventario. También guarda el id del movimiento generado.

4. Endpoints por entidad

En las siguientes secciones se usa la base **http://127.0.0.1:8000/api/v1**. En los ejemplos con identificadores se utilizan IDs reales del entorno de prueba que se compartieron durante el desarrollo. Cuando un endpoint crea un registro, el frontend debe guardar el id devuelto porque normalmente se utiliza en los pasos siguientes.

4.1 Empresa y autenticación

POST http://127.0.0.1:8000/api/v1/auth/register

Registra una nueva empresa y crea sus credenciales de acceso.

Qué se envía:

```
{
  "nombre": "PsiTwin Logistics",
  "ruc": "20123456789",
  "sector": "Logística",
  "correo": "admin@psitwin.com",
  "telefono": "987654321",
  "password": "123456"
}
```

Qué debe aparecer / qué devuelve: Los datos de la empresa creada y su identificador. La contraseña no debe mostrarse en el frontend después de registrarse.

POST http://127.0.0.1:8000/api/v1/auth/login

Inicia sesión con el correo y la contraseña de la empresa.

Qué se envía:

```
{
  "correo": "admin@psitwin.com",
  "password": "123456"
}
```

Qué debe aparecer / qué devuelve: Un token de acceso. El frontend debe conservarlo y enviarlo en las peticiones protegidas.

GET http://127.0.0.1:8000/api/v1/empresas/me

Obtiene el perfil de la empresa que está autenticada.

Qué debe aparecer / qué devuelve: Los datos de la empresa asociada al token actual.

PUT http://127.0.0.1:8000/api/v1/empresas/me

Actualiza los datos generales de la empresa autenticada.

Qué se envía:

```
{
  "nombre": "PsiTwin Logistics",
  "sector": "Logística y distribución",
  "correo": "admin@psitwin.com",
  "telefono": "987654321"
}
```

Qué debe aparecer / qué devuelve: La empresa con los datos actualizados.

PATCH http://127.0.0.1:8000/api/v1/empresas/me/password

Cambia la contraseña de la empresa autenticada.

Qué se envía:

```
{
  "password_actual": "123456",
  "password_nueva": "000000"
}
```

Qué debe aparecer / qué devuelve: Confirmación del cambio. El frontend debe pedir nuevamente la contraseña actual antes de enviar la operación.

DELETE http://127.0.0.1:8000/api/v1/empresas/me

Elimina la empresa autenticada.

Qué debe aparecer / qué devuelve: Confirmación de eliminación si la operación está permitida.

4.2 Almacén

POST http://127.0.0.1:8000/api/v1/almacenes

Crea un almacén perteneciente a una empresa.

Qué se envía:

```
{
  "id_empresa": "b39326db-fca6-4058-8194-8be9a15e8fd2",
  "nombre": "Almacén Principal",
  "ubicacion": "Lima, Perú",
  "area_m2": 500.0,
  "aforo": 40,
  "tipo_almacen": "General",
  "latitud": -12.046374,
  "longitud": -77.042793
}
```

Qué debe aparecer / qué devuelve: El almacén creado con su id. Ese id se usa después al crear zonas y operarios.

GET http://127.0.0.1:8000/api/v1/almacenes

Lista los almacenes registrados.

Qué debe aparecer / qué devuelve: Una lista de almacenes.

GET http://127.0.0.1:8000/api/v1/almacenes/df189bd9-0cf3-4e7c-90bf-f79dd8709381

Obtiene un almacén específico.

Qué debe aparecer / qué devuelve: Los datos del almacén solicitado.

GET http://127.0.0.1:8000/api/v1/almacenes/empresa/b39326db-fca6-4058-8194-8be9a15e8fd2

Lista los almacenes pertenecientes a una empresa.

Qué debe aparecer / qué devuelve: Una lista filtrada por empresa.

PUT http://127.0.0.1:8000/api/v1/almacenes/df189bd9-0cf3-4e7c-90bf-f79dd8709381

Actualiza los datos de un almacén.

Qué se envía:

```
{
  "nombre": "Almacén Principal Lima",
  "ubicacion": "Lima, Perú",
  "area_m2": 520.0,
  "aforo": 45,
  "tipo_almacen": "General",
  "latitud": -12.046374,
  "longitud": -77.042793
}
```

Qué debe aparecer / qué devuelve: El almacén actualizado.

DELETE http://127.0.0.1:8000/api/v1/almacenes/df189bd9-0cf3-4e7c-90bf-f79dd8709381

Elimina un almacén si las reglas del backend lo permiten.

Qué debe aparecer / qué devuelve: Confirmación de eliminación.

4.3 Zona

POST http://127.0.0.1:8000/api/v1/zonas

Crea una zona dentro de un almacén.

Qué se envía:

```
{
  "id_almacen": "df189bd9-0cf3-4e7c-90bf-f79dd8709381",
  "nombre": "Zona A",
```

```
  "tipo_zona": "Almacenamiento general",
  "temperatura_min": 18,
  "temperatura_max": 25,
  "capacidad": 1000
}
```

Qué debe aparecer / qué devuelve: La zona creada con su id.

GET http://127.0.0.1:8000/api/v1/zonas

Lista todas las zonas.

Qué debe aparecer / qué devuelve: Una lista de zonas.

GET http://127.0.0.1:8000/api/v1/zonas/{id_zona}

Obtiene una zona específica.

Qué debe aparecer / qué devuelve: Los datos de la zona indicada.

GET http://127.0.0.1:8000/api/v1/zonas/almacen/df189bd9-0cf3-4e7c-90bf-f79dd8709381

Lista las zonas de un almacén.

Qué debe aparecer / qué devuelve: Una lista de zonas pertenecientes al almacén.

PUT http://127.0.0.1:8000/api/v1/zonas/{id_zona}

Actualiza una zona.

Qué se envía:

```
{
  "nombre": "Zona A - Secos",
  "tipo_zona": "Secos",
  "temperatura_min": 18,
  "temperatura_max": 25,
  "capacidad": 1200
}
```

Qué debe aparecer / qué devuelve: La zona actualizada.

DELETE http://127.0.0.1:8000/api/v1/zonas/{id_zona}

Elimina una zona.

Qué debe aparecer / qué devuelve: Confirmación de eliminación.

4.4 Rack

POST http://127.0.0.1:8000/api/v1/racks

Crea un rack dentro de una zona.

Qué se envía:

```
{
  "id_zona": "<ID_ZONA>",
  "nivel": 1,
  "capacidad": 200,
  "estado": "DISPONIBLE"
}
```

Qué debe aparecer / qué devuelve: El rack creado con su id.

GET http://127.0.0.1:8000/api/v1/racks

Lista todos los racks.

Qué debe aparecer / qué devuelve: Una lista de racks.

GET http://127.0.0.1:8000/api/v1/racks/19420f3f-12a9-4f75-8f2f-5ff51883d911

Obtiene un rack específico.

Qué debe aparecer / qué devuelve: Los datos del rack indicado.

GET http://127.0.0.1:8000/api/v1/racks/zona/{id_zona}

Lista los racks de una zona.

Qué debe aparecer / qué devuelve: Una lista filtrada por zona.

PUT http://127.0.0.1:8000/api/v1/racks/19420f3f-12a9-4f75-8f2f-5ff51883d911

Actualiza capacidad, nivel o estado del rack.

Qué se envía:

```
{
  "nivel": 1,
  "capacidad": 220,
  "estado": "DISPONIBLE"
}
```

Qué debe aparecer / qué devuelve: El rack actualizado.

DELETE http://127.0.0.1:8000/api/v1/racks/19420f3f-12a9-4f75-8f2f-5ff51883d911

Elimina un rack.

Qué debe aparecer / qué devuelve: Confirmación de eliminación.

4.5 Producto

POST http://127.0.0.1:8000/api/v1/productos

Crea un producto en el catálogo.

Qué se envía:

```
{
  "id_empresa": "b39326db-fca6-4058-8194-8be9a15e8fd2",
  "id_rack": "19420f3f-12a9-4f75-8f2f-5ff51883d911",
  "nombre": "Producto de prueba",
  "categoria": "Alimentos",
  "familia": "Secos",
  "marca": "Marca Demo",
  "peso": 1.0,
  "volumen": 0.002,
  "stock": 0,
  "requiere_refrigeracion": false,
  "temperatura_min_almacenamiento": null,
  "temperatura_max_almacenamiento": null,
  "lead_time_dias": 3,
  "vida_util_dias": 180
}
```

Qué debe aparecer / qué devuelve: El producto creado con id_producto.

GET http://127.0.0.1:8000/api/v1/productos

Lista todos los productos.

Qué debe aparecer / qué devuelve: Una lista de productos.

GET http://127.0.0.1:8000/api/v1/productos/refrigerados

Lista solo productos que requieren refrigeración.

Qué debe aparecer / qué devuelve: Una lista de productos refrigerados.

GET http://127.0.0.1:8000/api/v1/productos/almacen/df189bd9-0cf3-4e7c-90bf-f79dd8709381

Lista los productos relacionados con un almacén.

Qué debe aparecer / qué devuelve: Una lista de productos del almacén.

GET http://127.0.0.1:8000/api/v1/productos/rack/19420f3f-12a9-4f75-8f2f-5ff51883d911

Lista los productos de un rack.

Qué debe aparecer / qué devuelve: Una lista de productos ubicados en ese rack.

GET http://127.0.0.1:8000/api/v1/productos/categoria/Alimentos

Lista los productos de una categoría.

Qué debe aparecer / qué devuelve: Una lista filtrada por categoría.

GET http://127.0.0.1:8000/api/v1/productos/85c09197-cc50-44de-8869-f53e8d103efb

Obtiene un producto específico.

Qué debe aparecer / qué devuelve: Los datos completos del producto.

PUT http://127.0.0.1:8000/api/v1/productos/85c09197-cc50-44de-8869-f53e8d103efb

Actualiza los datos del producto.

Qué se envía:

```
{
  "nombre": "Producto actualizado",
  "categoria": "Alimentos",
  "familia": "Secos",
  "marca": "Marca Demo",
  "peso": 1.1,
  "volumen": 0.002,
  "requiere_refrigeracion": false,
  "lead_time_dias": 4,
  "vida_util_dias": 180
}
```

Qué debe aparecer / qué devuelve: El producto actualizado.

DELETE http://127.0.0.1:8000/api/v1/productos/85c09197-cc50-44de-8869-f53e8d103efb

Elimina un producto si es posible.

Qué debe aparecer / qué devuelve: Confirmación de eliminación.

Para mostrar stock real en la interfaz, usar Inventario.stock_actual. El campo Producto.stock existe por compatibilidad del proyecto, pero no debería ser la fuente principal.

4.6 Inventario

POST http://127.0.0.1:8000/api/v1/inventarios

Crea el registro de stock de un producto en un rack.

Qué se envía:

```
{
  "id_producto": "85c09197-cc50-44de-8869-f53e8d103efb",
  "id_rack": "19420f3f-12a9-4f75-8f2f-5ff51883d911",
  "stock_actual": 100,
  "stock_minimo": 20,
  "stock_maximo": 200
}
```

Qué debe aparecer / qué devuelve: El inventario creado con id_inventario.

GET http://127.0.0.1:8000/api/v1/inventarios

Lista todos los inventarios.

Qué debe aparecer / qué devuelve: Una lista con producto, rack y niveles de stock.

GET http://127.0.0.1:8000/api/v1/inventarios/stock-bajo

Lista inventarios cuyo stock se encuentra bajo el nivel mínimo.

Qué debe aparecer / qué devuelve: Una lista útil para alertas y pantalla de reposición.

GET http://127.0.0.1:8000/api/v1/inventarios/sin-stock

Lista inventarios con stock igual a cero.

Qué debe aparecer / qué devuelve: Una lista de productos sin existencias.

GET http://127.0.0.1:8000/api/v1/inventarios/sobre-stock

Lista inventarios cuyo stock supera el máximo configurado.

Qué debe aparecer / qué devuelve: Una lista de casos de sobrestock.

GET http://127.0.0.1:8000/api/v1/inventarios/producto/85c09197-cc50-44de-8869-f53e8d103efb

Consulta el inventario asociado a un producto.

Qué debe aparecer / qué devuelve: Los registros de inventario del producto.

GET http://127.0.0.1:8000/api/v1/inventarios/rack/19420f3f-12a9-4f75-8f2f-5ff51883d911

Consulta el inventario de un rack.

Qué debe aparecer / qué devuelve: Los registros ubicados en ese rack.

GET http://127.0.0.1:8000/api/v1/inventarios/80b5c80c-c7c4-4ea3-aa55-e8f3889a6e67

Obtiene un inventario específico.

Qué debe aparecer / qué devuelve: Stock actual, mínimo, máximo, producto y rack.

PUT http://127.0.0.1:8000/api/v1/inventarios/80b5c80c-c7c4-4ea3-aa55-e8f3889a6e67

Actualiza la configuración del inventario y, si el backend lo permite, su stock actual.

Qué se envía:

```
{
  "stock_actual": 100,
  "stock_minimo": 20,
  "stock_maximo": 200
}
```

Qué debe aparecer / qué devuelve: El inventario actualizado.

DELETE http://127.0.0.1:8000/api/v1/inventarios/80b5c80c-c7c4-4ea3-aa55-e8f3889a6e67

Elimina un inventario.

Qué debe aparecer / qué devuelve: Confirmación. El backend puede rechazarlo si todavía existe stock.

4.7 Operario

POST http://127.0.0.1:8000/api/v1/operarios

Crea un operario para un almacén.

Qué se envía:

```
{
  "id_empresa": "b39326db-fca6-4058-8194-8be9a15e8fd2",
  "id_almacen": "df189bd9-0cf3-4e7c-90bf-f79dd8709381",
  "nombre": "Carlos",
  "apellido": "Pérez",
  "cargo": "Almacenero",
  "turno": "MAÑANA",
  "estado": "ACTIVO"
}
```

Qué debe aparecer / qué devuelve: El operario creado con id_operario.

GET http://127.0.0.1:8000/api/v1/operarios

Lista todos los operarios.

Qué debe aparecer / qué devuelve: Una lista de operarios.

GET http://127.0.0.1:8000/api/v1/operarios/empresa/b39326db-fca6-4058-8194-8be9a15e8fd2

Lista operarios de una empresa.

Qué debe aparecer / qué devuelve: Una lista filtrada por empresa.

GET http://127.0.0.1:8000/api/v1/operarios/estado/ACTIVO

Lista operarios según su estado.

Qué debe aparecer / qué devuelve: Una lista de operarios activos.

GET http://127.0.0.1:8000/api/v1/operarios/turno/MAÑANA

Lista operarios por turno.

Qué debe aparecer / qué devuelve: Una lista filtrada por turno.

GET http://127.0.0.1:8000/api/v1/operarios/48cc67d4-7f65-481f-b700-a93b2cdeeba0

Obtiene un operario específico.

Qué debe aparecer / qué devuelve: Los datos del operario.

PUT http://127.0.0.1:8000/api/v1/operarios/48cc67d4-7f65-481f-b700-a93b2cdeeba0

Actualiza los datos de un operario.

Qué se envía:

```
{
  "nombre": "Carlos",
  "apellido": "Pérez",
  "cargo": "Supervisor",
  "turno": "MAÑANA",
  "estado": "ACTIVO"
}
```

Qué debe aparecer / qué devuelve: El operario actualizado.

DELETE http://127.0.0.1:8000/api/v1/operarios/48cc67d4-7f65-481f-b700-a93b2cdeeba0

Elimina un operario.

Qué debe aparecer / qué devuelve: Confirmación de eliminación.

4.8 Movimiento

POST http://127.0.0.1:8000/api/v1/movimientos

Registra una entrada, salida o ajuste manual de inventario.

Qué se envía:

```
{
  "id_inventario": "80b5c80c-c7c4-4ea3-aa55-e8f3889a6e67",
  "id_operario": "48cc67d4-7f65-481f-b700-a93b2cdeeba0",
  "tipo_movimiento": "ENTRADA",
  "motivo_movimiento": "REPOSICION",
  "cantidad": 20,
  "observacion": "Ingreso de mercadería al almacén"
}
```

Qué debe aparecer / qué devuelve: El movimiento creado, incluyendo stock_anterior y stock_nuevo. El inventario queda actualizado automáticamente.

GET http://127.0.0.1:8000/api/v1/movimientos

Lista el historial de movimientos.

Qué debe aparecer / qué devuelve: Una lista cronológica de movimientos.

GET http://127.0.0.1:8000/api/v1/movimientos/30ff539e-7e81-4aab-a118-69dd8cfa318a

Obtiene un movimiento específico.

Qué debe aparecer / qué devuelve: El detalle completo del movimiento.

GET http://127.0.0.1:8000/api/v1/movimientos/inventario/80b5c80c-c7c4-4ea3-aa55-e8f3889a6e67

Lista movimientos de un inventario.

Qué debe aparecer / qué devuelve: Historial de entradas, salidas y ajustes del inventario.

GET http://127.0.0.1:8000/api/v1/movimientos/producto/85c09197-cc50-44de-8869-f53e8d103efb

Lista movimientos relacionados con un producto.

Qué debe aparecer / qué devuelve: Historial del producto.

GET http://127.0.0.1:8000/api/v1/movimientos/rack/19420f3f-12a9-4f75-8f2f-5ff51883d911

Lista movimientos relacionados con un rack.

Qué debe aparecer / qué devuelve: Historial de movimientos de esa ubicación.

GET http://127.0.0.1:8000/api/v1/movimientos/operario/48cc67d4-7f65-481f-b700-a93b2cdeeba0

Lista movimientos realizados por un operario.

Qué debe aparecer / qué devuelve: Historial filtrado por operario.

GET http://127.0.0.1:8000/api/v1/movimientos/tipo/ENTRADA

Lista movimientos de un tipo determinado.

Qué debe aparecer / qué devuelve: Por ejemplo, todas las ENTRADAS.

GET http://127.0.0.1:8000/api/v1/movimientos/motivo/RECEPCION

Lista movimientos por motivo.

Qué debe aparecer / qué devuelve: Por ejemplo, movimientos generados por recepciones.

Este endpoint debe confirmarse contra /docs si la versión actual del controlador no lo expone. El repositorio/servicio fue diseñado para consultar por motivo.

Reglas importantes: ENTRADA suma; SALIDA resta y debe rechazarse si no hay stock suficiente; AJUSTE establece el nuevo stock físico. Los movimientos no se editan ni se eliminan.

4.9 Proveedor

POST http://127.0.0.1:8000/api/v1/proveedores

Crea un proveedor de la empresa.

Qué se envía:

```
{
  "id_empresa": "b39326db-fca6-4058-8194-8be9a15e8fd2",
  "nombre": "Distribuidora Andina SAC",
  "ruc": "20123456789",
  "correo": "ventas@distribuidoraandina.com",
  "telefono": "987654321",
  "direccion": "Lima, Perú",
  "tiempo_entrega_estimado_dias": 3,
  "estado": "ACTIVO"
}
```

Qué debe aparecer / qué devuelve: El proveedor creado con id_proveedor.

GET http://127.0.0.1:8000/api/v1/proveedores

Lista todos los proveedores.

Qué debe aparecer / qué devuelve: Una lista de proveedores.

GET http://127.0.0.1:8000/api/v1/proveedores/empresa/b39326db-fca6-4058-8194-8be9a15e8fd2

Lista los proveedores de una empresa.

Qué debe aparecer / qué devuelve: Una lista filtrada por empresa.

GET http://127.0.0.1:8000/api/v1/proveedores/estado/ACTIVO

Lista proveedores por estado.

Qué debe aparecer / qué devuelve: Una lista de proveedores activos.

GET http://127.0.0.1:8000/api/v1/proveedores/{id_proveedor}

Obtiene un proveedor específico.

Qué debe aparecer / qué devuelve: Los datos del proveedor.

PUT http://127.0.0.1:8000/api/v1/proveedores/{id_proveedor}

Actualiza los datos editables del proveedor.

Qué se envía:

```
{
  "nombre": "Distribuidora Andina SAC",
  "correo": "ventas@distribuidoraandina.com",
  "telefono": "999888777",
  "direccion": "Lima, Perú",
  "tiempo_entrega_estimado_dias": 4,
  "estado": "ACTIVO"
}
```

Qué debe aparecer / qué devuelve: El proveedor actualizado.

DELETE http://127.0.0.1:8000/api/v1/proveedores/{id_proveedor}

Elimina un proveedor si es posible.

Qué debe aparecer / qué devuelve: Confirmación de eliminación.

4.10 Orden de reposición

POST http://127.0.0.1:8000/api/v1/ordenes-reposicion

Crea una nueva orden. El backend la inicia en estado PENDIENTE.

Qué se envía:

```
{
  "id_empresa": "b39326db-fca6-4058-8194-8be9a15e8fd2",
  "id_almacen": "df189bd9-0cf3-4e7c-90bf-f79dd8709381",
  "id_proveedor": "<ID_PROVEEDOR>",
  "fecha_estimada_recepcion": "2026-09-25T15:00:00Z",
  "origen": "MANUAL",
  "observacion": "Reposición de productos con stock bajo"
}
```

Qué debe aparecer / qué devuelve: La orden creada con id_orden_reposicion y estado PENDIENTE.

GET http://127.0.0.1:8000/api/v1/ordenes-reposicion

Lista todas las órdenes de reposición.

Qué debe aparecer / qué devuelve: Una lista de órdenes con sus estados.

GET http://127.0.0.1:8000/api/v1/ordenes-reposicion/empresa/b39326db-fca6-4058-8194-8be9a15e8fd2

Lista órdenes de una empresa.

Qué debe aparecer / qué devuelve: Una lista filtrada por empresa.

GET http://127.0.0.1:8000/api/v1/ordenes-reposicion/almacen/df189bd9-0cf3-4e7c-90bf-f79dd8709381

Lista órdenes de un almacén.

Qué debe aparecer / qué devuelve: Una lista filtrada por almacén.

GET http://127.0.0.1:8000/api/v1/ordenes-reposicion/proveedor/{id_proveedor}

Lista órdenes asociadas a un proveedor.

Qué debe aparecer / qué devuelve: Una lista filtrada por proveedor.

GET http://127.0.0.1:8000/api/v1/ordenes-reposicion/estado/PENDIENTE

Lista órdenes por estado.

Qué debe aparecer / qué devuelve: Una lista de órdenes PENDIENTES.

PATCH http://127.0.0.1:8000/api/v1/ordenes-reposicion/{id_orden}/aprobar

Aprueba una orden PENDIENTE.

Qué debe aparecer / qué devuelve: La orden con estado APROBADA. No necesita body.

PATCH http://127.0.0.1:8000/api/v1/ordenes-reposicion/{id_orden}/enviar

Marca como enviada una orden APROBADA.

Qué debe aparecer / qué devuelve: La orden con estado ENVIADA. No necesita body.

PATCH http://127.0.0.1:8000/api/v1/ordenes-reposicion/{id_orden}/cancelar

Cancela una orden que todavía puede cancelarse.

Qué debe aparecer / qué devuelve: La orden con estado CANCELADA. No necesita body.

GET http://127.0.0.1:8000/api/v1/ordenes-reposicion/{id_orden}

Obtiene una orden específica.

Qué debe aparecer / qué devuelve: Los datos de la orden y su estado actual.

PUT http://127.0.0.1:8000/api/v1/ordenes-reposicion/{id_orden}

Actualiza los datos editables de la orden, pero no fuerza estados de recepción.

Qué se envía:

```
{
  "fecha_estimada_recepcion": "2026-09-26T15:00:00Z",
  "observacion": "Entrega reprogramada"
}
```

Qué debe aparecer / qué devuelve: La orden actualizada.

DELETE http://127.0.0.1:8000/api/v1/ordenes-reposicion/{id_orden}

Elimina una orden solo cuando el backend lo permita, normalmente mientras está PENDIENTE.

Qué debe aparecer / qué devuelve: Confirmación de eliminación.

Estados que debe mostrar el frontend: PENDIENTE → APROBADA → ENVIADA → PARCIAL → RECIBIDA. PENDIENTE o APROBADA también pueden pasar a CANCELADA. PARCIAL y RECIBIDA se calculan a partir de las recepciones y no deberían ofrecerse como botones manuales.

4.11 Detalle de reposición

POST http://127.0.0.1:8000/api/v1/ordenes-reposicion/{id_orden}/detalles

Agrega un producto a una orden PENDIENTE.

Qué se envía:

```
{
  "id_producto": "85c09197-cc50-44de-8869-f53e8d103efb",
  "cantidad_solicitada": 100,
  "costo_unitario": 10.5
}
```

Qué debe aparecer / qué devuelve: La línea creada con id_detalle_reposicion.

GET http://127.0.0.1:8000/api/v1/ordenes-reposicion/{id_orden}/detalles

Lista los productos solicitados en una orden.

Qué debe aparecer / qué devuelve: Una lista con producto, cantidad solicitada y costo unitario.

GET http://127.0.0.1:8000/api/v1/detalles-reposicion/{id_detalle}

Obtiene una línea específica de la orden.

Qué debe aparecer / qué devuelve: El detalle de reposición.

PUT http://127.0.0.1:8000/api/v1/detalles-reposicion/{id_detalle}

Actualiza una línea mientras la orden siga PENDIENTE.

Qué se envía:

```
{
  "cantidad_solicitada": 120,
  "costo_unitario": 10.2
}
```

Qué debe aparecer / qué devuelve: El detalle actualizado.

DELETE http://127.0.0.1:8000/api/v1/detalles-reposicion/{id_detalle}

Elimina una línea mientras la orden siga PENDIENTE.

Qué debe aparecer / qué devuelve: Confirmación de eliminación.

4.12 Recepción de mercadería

POST http://127.0.0.1:8000/api/v1/recepciones

Crea una recepción para una orden ENVIADA o PARCIAL.

Qué se envía:

```
{
  "id_orden_reposicion": "<ID_ORDEN>",
  "id_operario": "48cc67d4-7f65-481f-b700-a93b2cdeeba0",
  "observacion": "Recepción de proveedor"
}
```

Qué debe aparecer / qué devuelve: La recepción creada con id_recepcion y fecha de recepción.

GET http://127.0.0.1:8000/api/v1/recepciones

Lista todas las recepciones.

Qué debe aparecer / qué devuelve: Una lista de recepciones registradas.

GET http://127.0.0.1:8000/api/v1/recepciones/orden/{id_orden}

Lista las recepciones de una orden.

Qué debe aparecer / qué devuelve: Una o varias recepciones asociadas a la orden.

GET http://127.0.0.1:8000/api/v1/recepciones/operario/48cc67d4-7f65-481f-b700-a93b2cdeeba0

Lista recepciones realizadas por un operario.

Qué debe aparecer / qué devuelve: Una lista filtrada por operario.

GET http://127.0.0.1:8000/api/v1/recepciones/{id_recepcion}

Obtiene una recepción específica.

Qué debe aparecer / qué devuelve: Los datos de la recepción.

4.13 Detalle de recepción

POST http://127.0.0.1:8000/api/v1/recepciones/{id_recepcion}/detalles

Registra las cantidades recibidas de una línea de la orden.

Qué se envía:

```
{
  "id_detalle_reposicion": "<ID_DETALLE_REPOSICION>",
  "id_inventario": "80b5c80c-c7c4-4ea3-aa55-e8f3889a6e67",
  "cantidad_recibida": 100,
  "cantidad_rechazada": 5,
  "motivo_rechazo": "Unidades dañadas"
}
```

Qué debe aparecer / qué devuelve: El detalle creado, incluyendo cantidad_aceptada = 95 e id_movimiento generado automáticamente. El inventario aumenta en 95 unidades.

GET http://127.0.0.1:8000/api/v1/recepciones/{id_recepcion}/detalles

Lista todos los detalles de una recepción.

Qué debe aparecer / qué devuelve: Una lista con cantidades recibidas, rechazadas, aceptadas e id_movimiento.

GET http://127.0.0.1:8000/api/v1/detalles-recepcion/{id_detalle}

Obtiene un detalle de recepción específico.

Qué debe aparecer / qué devuelve: El detalle completo de la recepción.

Si cantidad_rechazada es mayor que cero, debe enviarse motivo_rechazo. La cantidad rechazada no aumenta el inventario. El frontend no debe crear manualmente otro movimiento después de este POST.

5. Ejemplo completo del flujo de reposición

Este es el recorrido que debería seguir una interfaz cuando una empresa detecta que necesita reponer un producto. Se presenta paso a paso para mostrar qué id se obtiene en cada etapa y cómo se utiliza después.

1. Crear o seleccionar proveedor

El frontend registra un proveedor con POST /proveedores o selecciona uno existente. Debe guardar el id_proveedor devuelto.

2. Crear la orden

Con id_empresa, id_almacen e id_proveedor se crea la orden. El backend devuelve id_orden_reposicion y la deja PENDIENTE.

3. Agregar productos

Mientras la orden está PENDIENTE se agregan uno o varios detalles. Cada detalle devuelve id_detalle_reposicion.

4. Aprobar la orden

Cuando ya no se desea seguir editando el contenido, se llama al PATCH /aprobar. La orden pasa a APROBADA.

5. Marcarla como enviada

Cuando la orden ha sido enviada al proveedor, se llama al PATCH /enviar. La orden pasa a ENVIADA.

6. Crear la recepción

Cuando llega la mercadería, se crea RecepcionMercaderia usando id_orden_reposicion e id_operario. El backend devuelve id_recepcion.

7. Registrar lo recibido

Por cada producto recibido se hace POST a /recepciones/{id_recepcion}/detalles. Se envían id_detalle_reposicion, id_inventario, cantidad_recibida, cantidad_rechazada y motivo si corresponde.

8. Movimiento automático

El backend calcula la cantidad aceptada. Si es mayor que cero, crea automáticamente un Movimiento ENTRADA/RECEPCION y actualiza stock_actual del Inventario. El frontend solo debe refrescar la información.

9. Revisar inventario

El frontend puede volver a consultar GET /inventarios/{id_inventario} para mostrar el nuevo stock.

10. Revisar estado de orden

Se consulta GET /ordenes-reposicion/{id_orden}. Si aún falta mercadería, será PARCIAL. Cuando todo lo solicitado haya sido aceptado, será RECIBIDA.

6. Qué debería mostrar el frontend después de cada operación

Acción	Qué refrescar o mostrar
Crear almacén	Agregar el nuevo almacén al listado y conservar su id.
Crear zona	Actualizar las zonas del almacén seleccionado.

Acción	Qué refrescar o mostrar
Crear rack	Actualizar los racks de la zona seleccionada.
Crear producto	Agregar el producto al catálogo.
Crear inventario	Mostrar stock actual, mínimo y máximo para producto/rack.
Registrar movimiento	Refrescar stock del inventario y el historial de movimientos.
Crear orden	Mostrar estado PENDIENTE y habilitar edición de detalles.
Aprobar orden	Mostrar APROBADA y bloquear edición de detalles.
Enviar orden	Mostrar ENVIADA y habilitar el flujo de recepción.
Registrar detalle de recepción	Refrescar inventario, movimientos, cantidades recibidas de la orden y estado PARCIAL/RECIBIDA.

7. Errores que el frontend debe manejar

401 / 403: la sesión no es válida, el token falta o el usuario no tiene autorización. El frontend debe redirigir a login o mostrar un mensaje de acceso no permitido.

404: no se encontró el registro solicitado. Puede ocurrir si un id dejó de existir o se escribió incorrectamente.

409: una regla de negocio impide la operación. Ejemplos: stock insuficiente, intentar enviar una orden que todavía está PENDIENTE, recibir más unidades de las solicitadas o usar un operario inactivo.

422: el body no cumple las validaciones. El frontend debe revisar campos obligatorios, formatos y valores mínimos/máximos. Por ejemplo, cantidad_rechazada no puede ser mayor que cantidad_recibida.

500: indica un error no esperado del backend. No debe tratarse como error de datos del usuario; conviene mostrar un mensaje general y registrar el detalle para revisión.

8. Reglas que el frontend no debe romper

1. Usar **Inventario.stock_actual** como referencia principal del stock operativo.

2. No permitir editar ni borrar movimientos desde la interfaz. El movimiento funciona como historial.

3. Después de registrar un detalle de recepción, **no crear otro movimiento manual**. El backend ya genera ENTRADA/RECEPCION.

4. No permitir agregar o editar DetalleReposicion cuando la orden ya no está PENDIENTE.

5. No ofrecer botones para cambiar manualmente una orden a PARCIAL o RECIBIDA. Esos estados dependen de las cantidades aceptadas.

6. Para una recepción, utilizar un operario ACTIVO del mismo almacén de la orden.

7. Cuando se registra una SALIDA, mostrar correctamente el error si no existe stock suficiente.

9. Observaciones para la integración

La versión actual del proyecto ha pasado por algunos cambios de nombres y compatibilidad. En datos antiguos puede aparecer id en lugar de id_almacen, id_zona o id_rack. También hubo una versión anterior del estado de Rack con ACTIVO/INACTIVO y una versión actual con DISPONIBLE/LLENO/MANTENIMIENTO. El frontend debería basarse en los valores que exponga la documentación /docs del backend que esté ejecutándose.

También debe confirmarse en la integración final que el método utilizado por DetalleRecepcion para crear el movimiento automático coincide con el nombre real del servicio de movimientos. Desde la perspectiva del frontend esto no cambia el contrato esperado: al registrar un detalle de recepción con cantidad aceptada mayor que cero

debe devolverse id_movimiento y el inventario debe aparecer actualizado.

Por último, Producto todavía conserva un campo stock por compatibilidad, pero la arquitectura funcional del sistema separa el catálogo del producto del stock por ubicación. Por ello, para el dashboard y las pantallas de existencias, el frontend debe priorizar los endpoints de Inventario.

10. Checklist para empezar el frontend

• Implementar login y almacenamiento del token.

• Crear módulo Empresa/Perfil.

• Crear navegación Almacén → Zona → Rack.

• Crear catálogo de Productos.

• Crear pantalla de Inventario con filtros de stock bajo, sin stock y sobre stock.

• Crear módulo de Operarios.

• Crear historial de Movimientos y formulario para movimientos manuales permitidos.

• Crear módulo de Proveedores.

• Crear flujo de Orden de reposición con sus detalles.

• Mostrar acciones Aprobar, Enviar y Cancelar según el estado actual.

• Crear módulo de Recepciones y Detalles de recepción.

• Después de cada recepción, refrescar orden, inventario y movimientos.

• Validar errores 401, 404, 409 y 422 con mensajes claros para el usuario.

Con estas reglas y endpoints, el frontend puede construir el flujo completo sin necesidad de conocer la implementación interna del backend.